

An Enclaved Cache Coherence Design For Multicore Systems

Dennis Elorm Fleku

**Department of Electrical
Engineering and Computer
Science**

*Texas A&M University
Kingsville, US*

Abstract—As workloads for computing systems evolve, there will be a continuous need for high performance processing elements with low latency processing capabilities. Computing system architectural and organizational changes for multicore systems are centered on providing the tools through the instruction set specifications for efficient designs that can process data in short latency while meeting the performance constraints of their target systems. Also, consistent view of data across the many core devices, such as multicore scalar, pipelined, heterogeneous, multithreaded, simultaneous multithreaded, and chip multiprocessing systems, is imperative to provide correct execution results in program specified order and minimize the latency of overall program execution. Different cache coherence studies for multicore systems have been undertaken using software, and hardware approaches to maintain a coherent view of data across multicore systems, mostly coined around either the directory or snoop-based coherence schemes for cache memory coherence. This study presents a coherent design and model, ECCP model, for maintaining cache coherence in multicore systems by enhancing the snoop and directory protocol schemes which can be scaled for computing resource critical environments through an enclave scheme. It further assesses how the enclaved design provides short latency memory operations in the enclaved environment.

Index Items—*cache coherence, multi-level cache design, multi-core systems, enclaves, schemes, protocol, snoopy, directory*

I. INTRODUCTION

Modern multicore systems need a consistent view of data to carry out their processing tasks efficiently to offer performance benefits for their class of product families. Different approaches have been utilized to improve the performance of multicore systems such as the adoption of different processor configurations, banked memory technologies, and the use of high-speed buses for multiprocessor performance enhancements [1][2].

Each core in the multi-device system normally consists of a processor, multi-levels of local memory (cache), and a shared bus system to communicate with the other cores and the main memory. The hierarchy of the cache memories can range from level 1 (L1), level 2 (L2), to level (L3) depending on the architectural design considerations and the target market. This relieves the processor core of constant memory references with the memory performance lagging that of the processor.

The correctness of data in the core's local caches is fundamental to its operation and the results written to the architectural states of the core and, thus results seen by the programmer. The performance of data in cache and main

memory is an equally important design metric as its consistency states for data. As a result, cache coherence schemes have been developed to solve the consistency data issue across processor core caches for multicore systems with performance factored into their design.

Common cache coherence schemes for multicore have evolved from snoop-based schemes, directory approaches and the incorporation of software approaches [3][4]. The provision of directory protocols in the processor memory organization to 'cache' cache line state information in different caches takes a centralized approach for cache coherence. Snoopy schemes tend to provide real time information of cache data transactions using constant monitoring of a shared system bus. These two main schemes have pitfalls being high bus traffic for snoop-based protocols and bottlenecked single source for the directory scheme in large core systems. Variants of the cache coherence schemes have sought to provide different coherence protocols to the MESI, and MOESI protocols for high performance [5]. Other studies have considered a transactional scheme to maintain coherence on instruction boundaries for cache line data [6].

This study takes a design approach for cache coherence by detailing out an enclaved-based scheme for snooping relevant cache data state information to other processor cores in adjacent enclaves over a common bus system while decentralizing directory protocol across enclaves. The contribution of this study is as follows:

- i. Provision of an enclaved coherent design for cache coherence that can be scaled for large multicore systems.
- ii. Decentralization of directory protocol across enclaves to reduce directory protocol to per-enclave basis rather than per-core basis.
- iii. Introduction of an enclaved cache coherence protocol (ECCP), with the coherent design that forms the ECCP model, that minimizes memory operations within enclaves: modified cache line data can be transferred between enclave cores.
- iv. Enhanced snoopy protocol for message-passing across enclaves with less system bus traffic.

II. RELATED WORK AND MOTIVATION

A. Cache Coherence Protocols and Models

Cache coherence implies having a consistent view of data across processor core caches for a multiprocessor system for the following cache line data operations: read request by a core followed by a write request by another core, write request from one core followed by a read request from another requesting core, and synchronizing all write operations between cores [7].

Cache coherence protocols use cache line level information to encode cache line data state details in their individual cache controllers. These cache controllers are configured with protocols that monitor, update cache state information in other cores, and allow the use of cache line data across caches and processor cores. Their activities (cache controller) are usually centered around the coherence protocol installed.

The coherence protocol monitors activities, read and writes, of the cache line data in other controllers to cause state transitions for cache line entries to provide consistent data for the processor core to undertake its activities. The base cache coherence protocol is the MSI protocol with three states: M for Modified, S for Shared, and I for Invalid. (1) Invalid (I): a cache line state with data that cannot be used. (2) Shared (S): a cache line state for cache lines shared with caches, clean and can be referenced (read) by other cores. (3) Modified (M): a cache line state with cache line that is dirty or modified (compared to main memory) and is unique to the core. Only a single core cache can have a cache line in the Modified state for this scheme. Variants of this coherence protocol include introduce new states like the Exclusive (E): cache line state that is clean (unmodified with respect to main memory), contained only in the holding core and can be referenced (read) and Owned (O): cache line state that is dirty (modified with respect to main memory), and shared with other caches.

The coherence protocol in the controller operates in the following mode. For a newly copied memory block into the cache starts off with the E state. A subsequent read request READ from other cores would be granted by generating the coherent signal LREAD and transferring cache line to the requesting core. The cache line state would transition from the E state to the S state. A subsequent write request LWRITE from several other requesting cores would be granted to one core and the cache line state changed from S state to M state in the holding core. The cache line data would be invalidated in other cores if the coherence protocol is programmed with the WRITE INVALIDATE scheme or other the cache line state would be updated to the M state in all cores for the WRITE UPDATE scheme is used. But only one core cache would have the cache line data in the O state for the latter instance of a modified block.

Inclusiveness/Exclusiveness. Inclusive property for multi-level caches means cache lines in L1 caches (top level hierarchy) are also visible in L2 (low level hierarchy) caches. Other the hand, exclusive property means cache lines are resident in only one cache at a time. Other schemes use the exclusive property with low level cache hierarchies to store replaced cache lines.

Snoop Bus Protocols. Previous works have used bus snooping protocols to broadcast coherence signals across a

shared bus or split bus systems to other cores' controllers that may be susceptible to high bus traffic for large core systems [2].

Directory Protocols. Other studies have used directory-based protocols to maintain cache line data state information to orchestrate a cache coherence, with information exchange still over a shared bus. With this, a centralized directory is kept in memory acting as a single point of reference for all caches [5].

Back Invalidation. Inclusive last level caches (LLC) in multi-level cache systems incur additional memory access latencies if cache lines need to be replaced to service top level cache's requests or cache coherency requests from other cores necessitating references to main memory.

Shared Bus or Split Bus Systems. Studies have also harnessed a split bus system (with ORed wired signaling lines) to relay cache coherence signaling across cores.

This study uses a decentralized directory protocol with the snoop-based protocol over a shared system bus with a dedicated high speed connect between adjacent cores for data propagation to reduce system bus traffic with faster data transfer across cores.

III. ENCLAVED CACHE COHERENCE MODEL

We span out details on our cache coherence model consisting of Enclaved Cache Coherence Protocol (ECCP) and a coherent design.

A. Core Enclaves and Cache Hierarchy

Enclaves. We consider a multicore system made up of N cores (for the purposes of the study we assume $N = 4$) and segmented into enclaves: each enclave consists of 2 cores. Core enclaves are introduced to decentralize the distribution of directories among cores while reducing the cache controller directory count to per-enclave rather than per-core. Each core can assume a scalar single thread instruction issue type.

Cache Hierarchy. We consider a multi-level (3 level) cache design for our work, used in most modern processor cores [8][9]. The first level (L1) is a split cache: between instruction and data to allow parallelism in data processing operations. Levels 2 (L2) and 3 (L3) assume a unified cache implementation with L3 having the largest size. All level caches (L1, L2, and L3) use write-back policy and cache levels L1 & L2 have the cache inclusive property with L3 cache being exclusive. L2 and L3 caches maintain empty cache line entries for new requests, to remove the latency effects due to back propagation of replaced cache line data from top level caches and requests originating from other cores (in enclave or out enclave). For a set associative cache mapping implementation, the first and last sets are reserved for these purposes for caches L2 and L3. LLC cache (L3) is used as a backing store to receive and process requests to/from higher level caches.

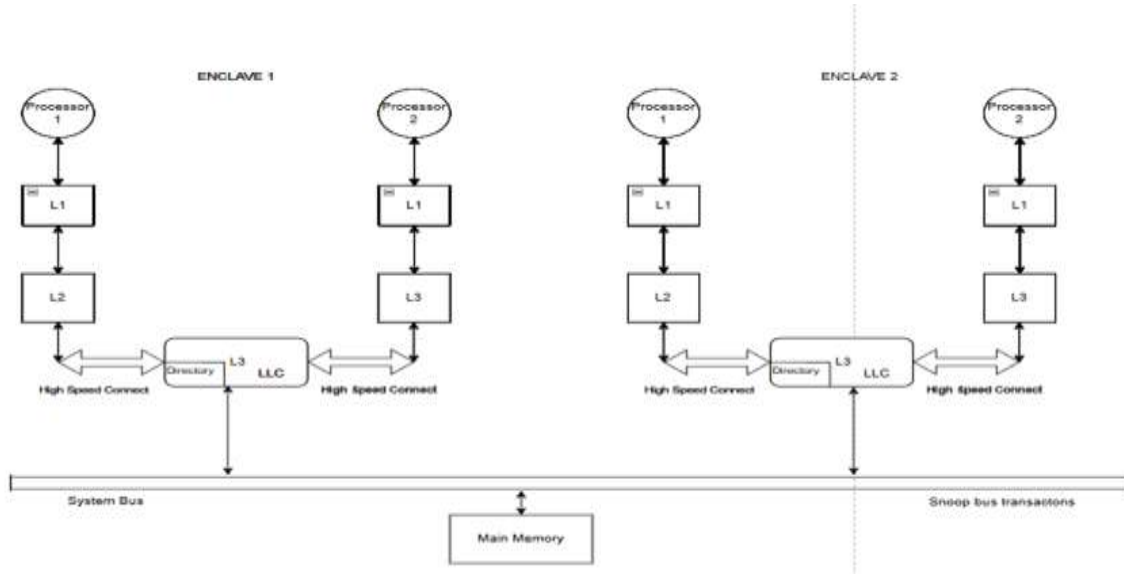


Fig. 1. Coherent design with 2 enclaves and 4 cores, 2 cores per enclave.

B. Enclave Coherence Directory

A unique feature of the ECCP model is its enclave coherence directory. It saves cache line state information for core caches in the enclave. This cache line state information can be referenced by L1 or L2 caches within the enclave (intra-enclave) or directories for other enclaves (inter-enclave). The shared L3 cache in an enclave contains the coherence directory connected to the enclave core's L2 cache using a dedicated high-speed connect.

C. Shared L3 caches and Inclusive/Exclusive cache

The shared L3 cache is the last level cache (LLC) with an exclusive property with respect to L3 caches in inter-enclave regions. It remains as a first point of call for requests trickling in from other enclaves or main memory. For instance, if a LREAD or LWRITE request from an intra-enclave core results in a broadcast request over the shared bus system to an inter-enclave directory or main memory, the received cache line needs to be updated in both the L3 cache and the requesting core's cache.

D. Cache Line Tag Bits

The enclave directory contains the state information of cache lines present in cores within the enclave. The state information can be encoded as additional bits using the naming convention ECC bits: Enclave: Core: Cache level. Two bits for the enclave, a bit for both the core and cache level. The ECC bits are used in conjunction with the Modified (M), Referenced (R), Present/Absent, and Cache Disabled (CD) bits to track cache line data across intra-enclave and inter-enclave caches.

E. Cache Signal Lines and Coherence Control Unit

Coherency signaling is achieved using the system data bus and the introduced high-speed link connecting the LLC and main memory and L2 and L3 caches, respectively for a processor core.

A coherence control unit (CCU) positioned in the shared L3 caches in the enclave region controls intra-core request streams between L1, L2, and L3 caches and the inter-enclave directory messaging (and with main memory) in a counter-based round-robin time sliced manner. Requests are serviced in oldest first sequence to ensure references to memory are in program order. Each enclave takes turn to propagate requests from their buffers to other enclaves or main memory i.e. enclave directory-to-enclave directory or enclave directory-to-main memory with on-chip L1 to L2 caches message-passing streams occurring parallelly.

F. Main Memory

The multicore system has a main memory that can be accessed using the system bus. The shared L3 cache (directory) connects to the main memory using the system bus. Both the main memory and L3 (LLC) cache use a buffer to keep outstanding requests and service requests (ordering of requests) using the oldest first approach. Main memory is banked according to enclaves, multi-ported for parallel enclave memory request servicing.

IV. ENCLAVE CACHE COHERENCY PROTOCOL (ECCP)

To present how the ECCP protocol works, we first expand on the MOESI protocol as the baseline protocol and thereof note the differences with respect to our ECCP protocol.

The MOESI protocol works on a per-core basis while the ECCP protocol can be configured for operation on per-enclave basis. ECCP has similar active states (like MOESI) but can transfer modified cache lines across cores (caches) in an enclave without main memory update and granting them write capabilities. Its active states are as follows: Modified (M), Owned (O), Shared (S), Exclusive (E), and Invalid (I). For ECCP, the Owned (O) state indicates a cache line either clean or dirty is shared with another processor core cache.

For an operational model, newly copied cache lines have an initial E-state and transitions to S state in the requesting core

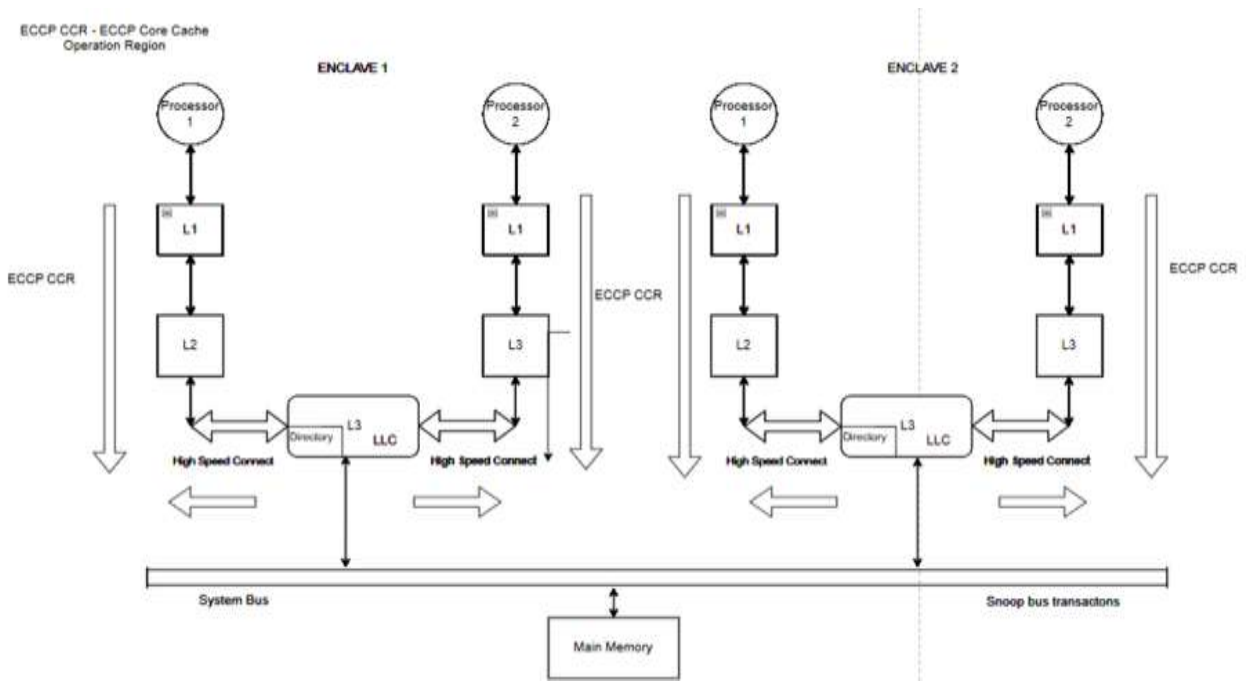


Fig. 2. ECCP core cache operation regions in the enclave.

and O state in the holding core once it receives a LREAD request from another (requesting) core. A LWRITE request from a requesting core transitions a cache line data state from either S or E or O states in the holding cores to M state in the requesting core and I state in the holding core.

The working of the ECCP protocol is further discussed in the following categories: ECCP Core Cache Operation, ECCP Intra-Enclave Operation, and ECCP Inter-Enclave Operation

A. ECCP Core Cache Operation

The ECCP Core Cache operation depicts the interaction between L1 and L2 caches:

The L1 and L2 caches are write back inclusive caches. Requests characteristically trickle in from the processor core's processing element to the L1 cache down to the L2 cache. Load operations with LREAD cache line request first check the L1 cache for a hit (if not a L1 hit it propagates request to L2 cache). On an L2 cache miss, L2 sends request over its dedicated high-speed link to L3 cache. Store operations infer a LWRITE coherence request that modifies L1 and L2 cache line data entries or sends a LWRITE request to L3 cache.

In effect, LREAD requests can be serviced from L1 cache, or L2 cache resulting in the following transitions: E-state in the holding cache to S state in the requesting cache and O state in the holding cache, and M state in the requesting cache to M state in the holding cache for LREAD requests. LWRITE requests have the transitions: E or S or O states in the holding cache to M state in the requesting cache and M state in all holding caches. LWRITE request modifies both L1 and L2 cache line data. On a cache miss in L1 or L2 caches, requests are sent to L3 cache and received cache line data updates both L1 and L2 caches.

B. ECCP Intra-Enclave Operation

The ECCP Intra-Enclave operation depicts interaction between L3 cache, the processor core's L2 caches (connected over a high speed connect), and the coherence directory. The shared L3 cache can receive a LREAD or LWRITE request from the cores in the enclave (L2 caches). While the enclave directory can also service request for the enclave's processor core L2 cache and enclave directories for other enclaves. Requests that propagate to the enclave directory does a look-up in the enclave directory to find the visibility information of the cache line data in other cores in the enclave. LREAD requests have the following transitions: E-state in the holding cache to O state in the holding cache and S state in the requesting cache. S state or M state cache line data have state information maintained in both the holding and requesting caches. LWRITE requests have the following transitions: E or M or O states in the holding cache to I state in the holding caches (other caches having the cache line data) and M state in the requesting cache.

C. ECCP Inter-Enclave Operation

The ECCP Inter-Enclave operation depicts interaction between intra-enclave, inter-enclave coherence directories and main memory. They characterize requests that propagate to the enclave directory and need to be serviced from an inter-enclave directory (different enclave) or main memory.

The enclave directory in the LLC keeps track of cache lines in other cores and enclaves. It periodically snoops the system bus to monitor request streams from other enclaves (controlled the coherence control unit) and responds to request streams. LREAD requests can be serviced directly over the system bus while LWRITE requests (LREAD modified cache line data) to an inter-core enclave directory needs to update main memory with the requested cache line data with the new write data before receiving a copy of the cache line (requests that are serviced directly from main memory are similarly updated and copied). This allows

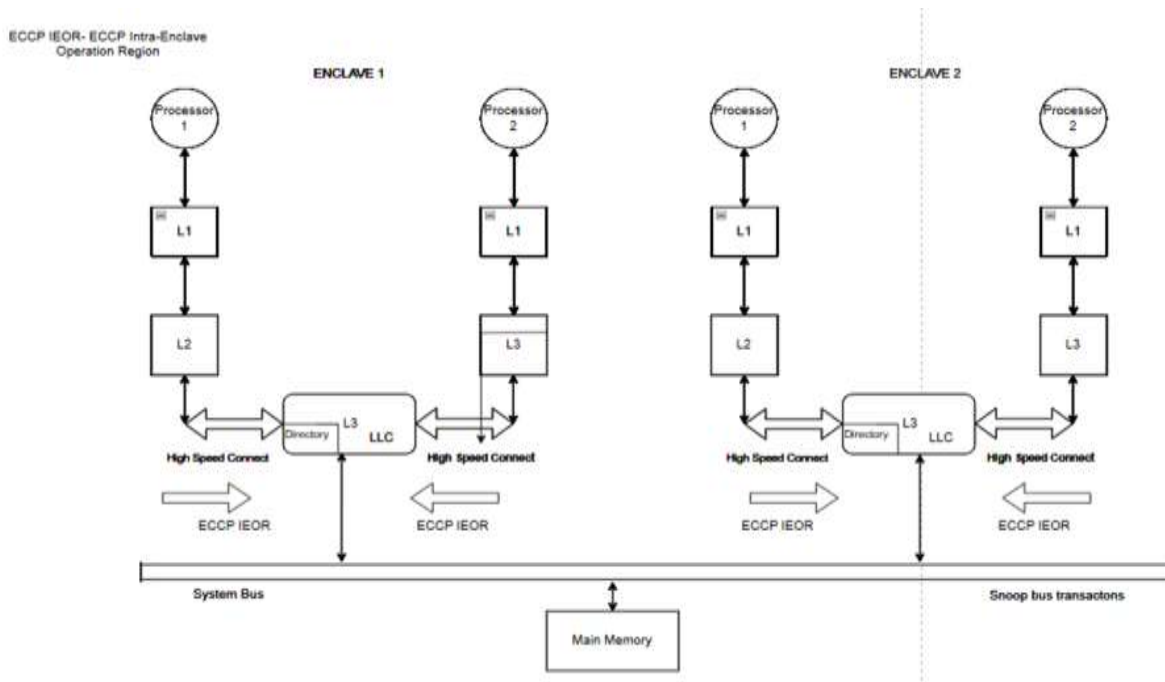


Fig. 3. ECCP intra-enclave operation regions.

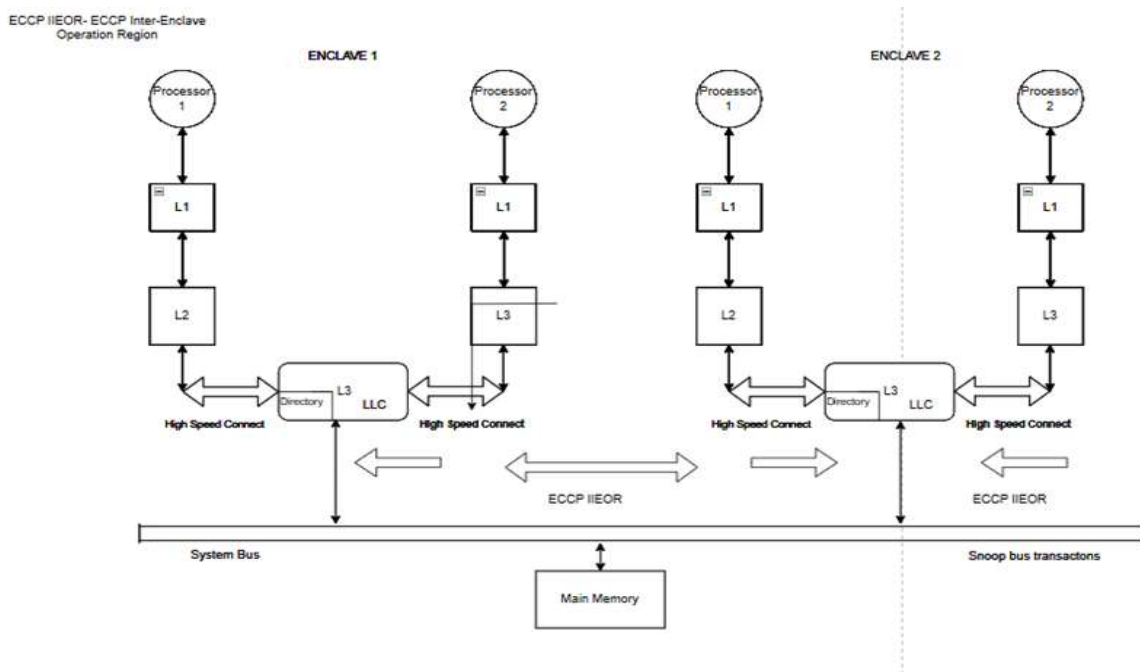


Fig. 4. ECCP inter-enclave operation regions.

modified cache lines to be transferred between cores in the different enclaves and main memory with updates reflected to main memory before sent to the requesting core. LREAD requests have the following transitions: E-state in the holding cache to O state in the holding cache and S state in the requesting cache, O state in the holding cache to O state in the holding cache and S state in the requesting cache, and M state in the holding cache to I state in all holding caches and E-state in the requesting cache. LWRITE requests have the following transitions: E or M or O or S states in the holding cache to I state in the holding caches (other caches having the cache line data) and E-state in the requesting cache.

D. Cache Line Replacement

Cache lines replaced in L1 or L2 caches are copied to the LLC (L3). In the LLC, replaced cache lines are copied to main memory if in a M-state or otherwise discarded. Replaced cache lines in O state in the LLC transfer cache line ownership to a different cache (intra or inter - enclave) that contains the cache line data. Cache lines in the M state, invalidates all other copies of the cache line in other caches (intra or inter - enclave) upon replacement. This is synonymous to LWRITE request for cache line data for ECCP Intra-Enclave Operation and LREAD request for a Modified cache line for ECCP Intra-Enclave Operation.

E. Cache Line State Transitions for ECCP

We present state transitions of ECCP for the exclusive, modified, shared, and owned states. The invalid state has LREAD or LWRITE requests as cache misses, so its state transitions are omitted. For all the states depicted, LWRITE request in L1 and L2 caches updates the holding cache, and the requesting cache states to M state and invalidates all other cache line entry in other caches. The state transitions for exclusive, modified, shared, and owned states are shown in Tables I, II, III and IV respectively.

F. ECCP Model Performance Analysis

The ECCP model, protocol and design, provides a host of benefits for multicore systems. We elaborate on how they can enhance multicore systems implementation wise.

Decentralized Directory Scheme. ECCP is centered on a decentralized directory coherence scheme that distributes directory look-up across enclaves while reducing the directories count to per-enclave basis rather than per-core basis. This allows fast look-up in enclave directories for faster cache hits (low latency look-ups) while enhancing performance.

Enclave Snoop Scheme. ECCP model allows snoop-based operation between enclaves and enclave-main-memory. The system bus snoop occurs only when requests need to be transferred between enclave entities. This reduces system bus traffic and allows scaling of the ECCP model for new added enclaves for a particular system bus bandwidth.

Dedicated High Speed Connect. Dedicated high speed connects between intra-enclave L2 caches and LLC caches minimizes system bus traffic and cache hit latency while providing faster lookups in enclave directory

Enhanced Cache Line Writes. ECCP relies on the enclave segmentation to allow transfer of modified cache lines data between cores (caches) in the same enclave. This feature enhances processor level “store operations” in that write operations can update cache lines without referencing main memory (a diversion from the MOESI functionality). Write operations can occur in minimized latency for a cache line hit.

Combined Directory and Snoop based Protocols. ECCP offers the combined performance of both directory and snoop-based coherence protocols while reducing their side effects: centralized look-up for cache lines in directory (to decentralized directory scheme) with cache line information relay over a shared bus (periodic snoop bus traffic for buffered requests).

G. ECCP Performance Model

The ECCP provides the benefit of allowing transfer of modified cache line data between cores in an enclave. This minimizes cache hit latency over the dedicated high speed connect between processor core caches, specifically L1 and L2 caches.

Assuming E = number of enclaves, N = number of cores, I = total of requests in an enclave, 50% cache hit ratio with efficient dependency check hardware and branch prediction schemes, main memory latency = S , cache memory latency with ECCP protocol = P , and enclave factor and with bus overheads = E/N

Lemma 1. Without Enclave factor

ECCP Performance Metric for an enclave = $I * 0.5 * P$

Lemma 2. With Enclave factor

ECCP Performance Metric for an enclave = $I * 0.5 * P * E / N$

H. ECCP Python Model

A Python Model is developed for the ECCP model which consists of the Memory model, Cache model, Core model, and Enclave model to ascertain the performance of the ECCP model. We simulate a series of writes to a location in cache memory and main memory to depict cache line hit for processor programs and the ECCP model’s ability to allow transfer of modified cache lines in enclaves and main memory (without ECCP model intra-enclave cache line transfer).

Taking the weighted performance metric ratio of ECCP cache reference latency to main memory reference latency, we record about 80% latency improvement for cache memory references in the enclave. The results demonstrate the minimized cache line latency introduced due to the ECCP model (transfer of modified blocks in the enclave for faster data retrieval) which eliminates main memory reference for cache line hits in the enclave.

```
## ECCP model
import time, random

# Memory Model
class Memory:
    mem_data = {}

    def __init__(self):
        self.data = None
        self.address = None

    def mem_count(self):
        count = random.randint(100000000, 10000000000)
        while count:
            count -= 1

    def write_mem(self, address, data):
        self.mem_count()
        Memory.mem_data[address] = data

    def read_mem(self, address):
        self.mem_count()
        return Memory.mem_data[address]

    def time_cnt(self):
        return time.time_ns()

# Cache Model
class Cache:
    cache_data = {}

    def __init__(self):
        self.cache_index = 0
        self.data = None
        self.address = None
        self.set_cache_index()

    def set_cache_index(self):
```



```

    if self.cache_index <= 2:
        self.cache_index += 1

def get_cache_index(self):
    return self.cache_index

def cache_count(self):
    count = random.randint(800000, 1000000)
    while count:
        count -= 1

def write_cache(self, address, data):
    self.cache_count()
    Cache.cache_data[address] = data

def read_cache(self, address):
    self.cache_count()
    return Cache.cache_data[address]

def time_cnt(self):
    return time.time_ns()

# Processor Core Model
class Core:

    def __init__(self):
        self.core_index = 0
        self.cache = Cache()
        self.set_core_index()

    def set_core_index(self):
        if self.core_index <= 2:
            self.core_index += 1

    def get_core_index(self):
        return self.core_index

# Enclave Model
class Enclave:

    def __init__(self):
        self.enclave_index = 0
        self.core = Core()
        self.set_enclave_index()

    def set_enclave_index(self):
        if self.enclave_index <= 4:
            self.enclave_index += 1

    def get_enclave_index(self):
        return self.enclave_index

    def init_core(self):
        self.core.set_core_index()
        return self.core.get_core_index()

    def init_cache(self):
        self.core.cache.set_cache_index()
        return self.core.cache.get_cache_index()

```

V. CONCLUSIONS

This study presented an overview of cache coherence in multicore systems through an Enclave Cache Coherence Protocol (ECCP) model, a cache coherence design and protocol. The presented coherent design segments processor cores into enclave regions for decentralization of the directory coherence protocol across enclaves to minimize look-up latency associated with centralized directory schemes that mostly positions directory protocol in main memory, introducing latency overheads. Also, it uses a high speed connect between core L2 cache and a shared L3 cache in the enclave to provide message passing between cores in an enclave for faster cache hits with minimized memory latency that is associated main memory access through the shared system bus. The ECCP model also utilizes the snoop-based protocol scheme to transact relevant messages between enclaves for less system bus traffic in addition to scalability of the multicore system i.e. creation of new enclaves for a specific system bus bandwidth.

The study further introduces an ECCP coherence protocol, a variant of the MOESI protocol that monitors cache line data state information across cores within enclaves (intra-enclave), and in other enclave regions (inter-enclaves). The ECCP protocol enhances, the MOESI active states, memory operations (LOAD and STORE memory referencing) in enclaves by allowing the transfer of modified cache lines in enclaves using the dedicated high speed connect until it is replaced or requested by a different enclave. This feature of the ECCP model was assessed with a Python ECCP model for a series of memory write simulations and recorded approximately 80% latency reduction due to faster cache line data retrieval in the enclave cores compared to main memory references.

While the study offered a coherent cache design and protocol that can be used in modern processors to achieve cache coherence, it can be improved for seamless integration into heterogenous processor environments that handle computations for varied workloads and further assessed in terms of the ECCP model's capabilities in maximizing cache hit ratio in enclaves, minimizing main memory latency, and determining the maximum allowable degree of scaling of enclave regions for the different workloads for a cache coherent multicore system.

TABLE I. EXCLUSIVE (E) STATE TRANSITIONS

Cache State	line	Request	Requesting Cache	Holding Cache	Transitioned State (Requesting Cache)	Transitioned State (Holding Cache)
Exclusive		LREAD	L1 cache	L1 cache	Exclusive	Exclusive
Exclusive		LREAD	L1 cache	L2 cache	Shared	Owned
Exclusive		LREAD	L2 cache	L3 cache	Shared	Owned
Exclusive		LREAD	L2 cache	Enclave directory (intra-core)	Shared	Owned
Exclusive		LREAD	L2 cache	Enclave directory	Shared	Owned
Exclusive		LREAD	Enclave directory (intra-core)	Enclave directory	Shared	Owned
Exclusive		LREAD	Enclave directory (intra-core)	L2 cache	Shared	Owned
Exclusive		LREAD	Enclave directory (intra-core)	L3 cache	Shared	Owned
Exclusive		LREAD	Enclave directory	Enclave Directory	Shared	Owned
Exclusive		LWRITE	L1 cache	L1 cache	Modified	Modified
Exclusive		LWRITE	L1 cache	L2 cache	Modified	Owned
Exclusive		LWRITE	L2 cache	L3 cache	Modified	Owned
Exclusive		LWRITE	L2 cache	Enclave directory (intra-core)	Modified	Owned
Exclusive		LWRITE	L2 cache	Enclave directory	Exclusive	Invalid
Exclusive		LWRITE	Enclave directory (intra-core)	Enclave directory	Exclusive	Invalid
Exclusive		LWRITE	Enclave directory (intra-core)	L2 cache	Modified	Owned
Exclusive		LWRITE	Enclave directory (intra-core)	L3 cache	Modified	Owned
Exclusive		LWRITE	Enclave directory	Enclave Directory	Modified	Owned

TABLE II. MODIFIED (M) STATE TRANSITIONS

Cache State	line	Request	Requesting Cache	Holding Cache	Transitioned State (Requesting Cache)	Transitioned State (Holding Cache)
Modified		LREAD	L1 cache	L1 cache	Modified	Modified
Modified		LREAD	L1 cache	L2 cache	Modified	Modified
Modified		LREAD	L2 cache	L3 cache	Modified	Modified
Modified		LREAD	L2 cache	Enclave directory (intra-core)	Modified	Modified
Modified		LREAD	L2 cache	Enclave directory	Exclusive	Invalid
Modified		LREAD	Enclave directory (intra-core)	Enclave directory	Exclusive	Invalid
Modified		LREAD	Enclave directory (intra-core)	L2 cache	Modified	Modified
Modified		LREAD	Enclave directory (intra-core)	L3 cache	Modified	Modified
Modified		LREAD	Enclave directory	Enclave Directory	Exclusive	Invalid
Modified		LWRITE	L1 cache	L1 cache	Modified	Modified
Modified		LWRITE	L1 cache	L2 cache	Modified	Modified
Modified		LWRITE	L2 cache	L3 cache	Modified	Invalid
Modified		LWRITE	L2 cache	Enclave directory (intra-core)	Modified	Invalid
Modified		LWRITE	L2 cache	Enclave directory	Exclusive	Invalid
Modified		LWRITE	Enclave directory (intra-core)	Enclave directory	Exclusive	Invalid
Modified		LWRITE	Enclave directory (intra-core)	L2 cache	Modified	Invalid
Modified		LWRITE	Enclave directory (intra-core)	L3 cache	Modified	Invalid
Modified		LWRITE	Enclave directory	Enclave Directory	Exclusive	Invalid

TABLE III. SHARED (S) STATE TRANSITIONS

Cache State	line	Request	Requesting Cache	Holding Cache	Transitioned State (Requesting Cache)	Transitioned State (Holding Cache)
Shared		LREAD	L1 cache	L1 cache	Shared	Shared
Shared		LREAD	L1 cache	L2 cache	Shared	Shared
Shared		LREAD	L2 cache	L3 cache	Shared	Shared
Shared		LREAD	L2 cache	Enclave directory (intra-core)	Shared	Shared
Shared		LREAD	L2 cache	Enclave directory	Shared	Shared
Shared		LREAD	Enclave directory (intra-core)	Enclave directory	Shared	Shared
Shared		LREAD	Enclave directory (intra-core)	L2 cache	Shared	Shared
Shared		LREAD	Enclave directory (intra-core)	L3 cache	Shared	Shared
Shared		LREAD	Enclave directory	Enclave Directory	Shared	Shared
Shared		LWRITE	L1 cache	L1 cache	Modified	Modified
Shared		LWRITE	L1 cache	L2 cache	Modified	Modified
Shared		LWRITE	L2 cache	L3 cache	Modified	Invalid
Shared		LWRITE	L2 cache	Enclave directory (intra-core)	Modified	Invalid
Shared		LWRITE	L2 cache	Enclave directory	Exclusive	Invalid
Shared		LWRITE	Enclave directory (intra-core)	Enclave directory	Exclusive	Invalid
Shared		LWRITE	Enclave directory (intra-core)	L2 cache	Modified	Invalid
Shared		LWRITE	Enclave directory (intra-core)	L3 cache	Modified	Invalid
Shared		LWRITE	Enclave directory	Enclave Directory	Exclusive	Invalid

TABLE IV. OWNED (O) STATE TRANSITIONS

Cache State	line	Request	Requesting Cache	Holding Cache	Transitioned State (Requesting Cache)	Transitioned State (Holding Cache)
Owned		LREAD	L1 cache	L1 cache	Shared	Owned
Owned		LREAD	L1 cache	L2 cache	Shared	Owned
Owned		LREAD	L2 cache	L3 cache	Shared	Owned
Owned		LREAD	L2 cache	Enclave directory (intra-core)	Shared	Owned
Owned		LREAD	L2 cache	Enclave directory	Shared	Owned
Owned		LREAD	Enclave directory (intra-core)	Enclave directory	Shared	Owned
Owned		LREAD	Enclave directory (intra-core)	L2 cache	Shared	Owned
Owned		LREAD	Enclave directory (intra-core)	L3 cache	Shared	Owned
Owned		LREAD	Enclave directory	Enclave Directory	Shared	Owned
Owned		LWRITE	L1 cache	L1 cache	Modified	Modified
Owned		LWRITE	L1 cache	L2 cache	Modified	Modified
Owned		LWRITE	L2 cache	L3 cache	Modified	Invalid
Owned		LWRITE	L2 cache	Enclave directory (intra-core)	Modified	Invalid
Owned		LWRITE	L2 cache	Enclave directory	Exclusive	Invalid
Owned		LWRITE	Enclave directory (intra-core)	Enclave directory	Exclusive	Invalid
Owned		LWRITE	Enclave directory (intra-core)	L2 cache	Modified	Invalid
Owned		LWRITE	Enclave directory (intra-core)	L3 cache	Modified	Invalid
Owned		LWRITE	Enclave directory	Enclave Directory	Modified	Invalid

ACKNOWLEDGMENT

This work was supported by the publications and materials, repositories referenced, and it is purely for academic purposes.

VI. REFERENCES

- [1] O. Mutlu, "Rethinking Memory System Design," in *Mobile System Technologies Workshop*, 2016.
- [2] N. Henry, A. Krishnamurthy, H. M. Levy, L. Rizzo, S. Khan and B. E. Stephens, "CC-NIC: a Cache-Coherent Interface to the NIC," in *ASPLOS '24*, 2024.
- [3] X. Wang, Z. Wu, R. Pellizzoni and H. Patel, "Exclusive Hierarchies for Predictable Sharing in Last-level Cache," in *IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, May 2024.
- [4] Z. Fu, M. Wang, Z. Y and Z. Tan, "Cache Coherent Framework for RISC-V Many-core Systems," Co-located with ISCA, June 2023.
- [5] H. M. Shukur, S. R. M. Zeebaree, R. R. Zebari, O. M. Ahmed, L. M. Haji and D. M. Abdulqad, "Cache Coherence Protocols in Distributed Systems," 2020.
- [6] L. Hammond, V. Wong, M. Chen, B. D. Carlstrom, J. D. Davis, B. Hertzberg, M. K. Prabhu, H. Wijaya, C. Kozyrakis and K. Olukotun, "RETROSPECTIVE: Transactional Memory Coherence and Consistency," in *Stanford University*, 2023.
- [7] Z. A. Waisi and M. A. Agyeman, "An Overview of On-Chip Cache Coherence Protocols," in *Intelligent Systems Conference*, University of Northampton, UK, September 2017.
- [8] E. Cui and e. al, "RISC-V Instruction Set Architecture Extensions: A Survey," 2023.
- [9] NVIDIA, "Blackwell Architecture Technical Brief Powering the New Era of Generative AI and Accelerated Computing," [Online]. Available: <https://resources.nvidia.com/en-us-blackwell-architecture>. [Accessed December 2024].